

计算机图形学第九次实验报告

2312966 林晖鹏

一、实验要求

使用系统中提供的网格拓扑编写网格处理的算法代码，实现Loop细分与 $\sqrt{3}$ 细分方法，对比结果并进行分析

二、实验环境

直接使用的学校实验室的电脑，这里环境配置不多赘述。

三、Loop细分算法实现

主要流程就三个：

1. 更新原始顶点

邻域遍历：使用 `MeshTopology` 中的 `findVtxNeighbors` 辅助函数获取顶点的一环邻域；

边界检查：使用 `isBoundary(v)` 检查顶点是否位于边界；

权重计算：根据顶点类型（内部/边界）采用对应公式计算新位置；

新位置存储在一个新的SDVertex中，并通过child指针链接到原始顶点。

2. 生成边顶点

对于网格中的每条边e，执行以下操作：

新顶点创建并计算权重：在边上创建一个新顶点，根据边的类型（内部/边界）采用对应公式计算新位置；

内部边（左）和边界边（右）权重计算：内部边由两个面共享，设端点为 v_0 、 v_1 ，两个面中的相对顶点为 v_2 、 v_3 ，而边界边只有一个相邻面，新位置即为中点，两个公式如下：

$$v_{edge} = \frac{3}{8}(v_0 + v_1) + \frac{1}{8}(v_2 + v_3), \quad v_{edge} = \frac{1}{2}(v_0 + v_1)$$

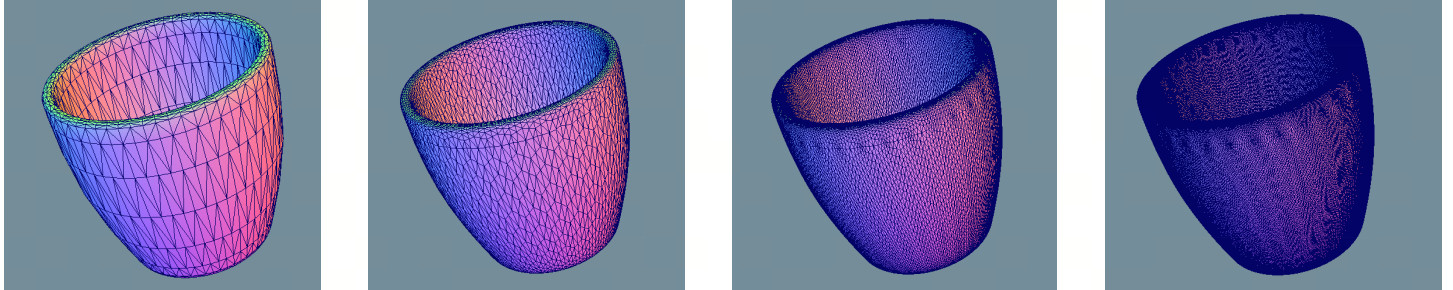
存储：新顶点存储在相邻面的sdverts数组中。

3. 重构网格

对于原始网格中的每个面，执行以下操作：

- 从原始顶点的child指针中获取更新后的原始顶点 (v'_0, v'_1, v'_2) ;
- 从面的sdverts数组中获取新的边顶点 (ep_0, ep_1, ep_2) 。注意 ep_0 位于与 v_0 相对的边上（即边 v_1-v_2 ），依此类推；
- 创建四个新面，确保所有面都保持逆时针 (CCW) 顺序：

最终效果如下：



四、 $\sqrt{3}$ 细分算法实现

主要流程同样是有三个：

1. 生成面中心顶点

中心顶点创建：对原始网格中的每个面，计算该面所有顶点的平均值作为面中心新顶点；

中心顶点存储：将生成的面中心顶点单独存储，建立与所属原始面的关联；

2. 更新原始顶点与边重构

对于网格中的每个原始顶点 v 和每条原始边 e ，执行以下操作：

原始顶点位置更新：根据顶点的一环邻域顶点及面中心顶点计算新位置，公式如下：

$$v_{new} = (1 - n\alpha_n)v + \alpha_n \sum_{i=0}^{n-1} v_i$$

边重构与拓扑更新：删除原始边，建立原始顶点与对应面中心顶点的新连接，形成新的三角形面；

边界处理：边界顶点仅根据其边界邻域顶点更新位置，边界边保留并调整连接关系，确保边界拓扑完整性。

3. 重构网格拓扑

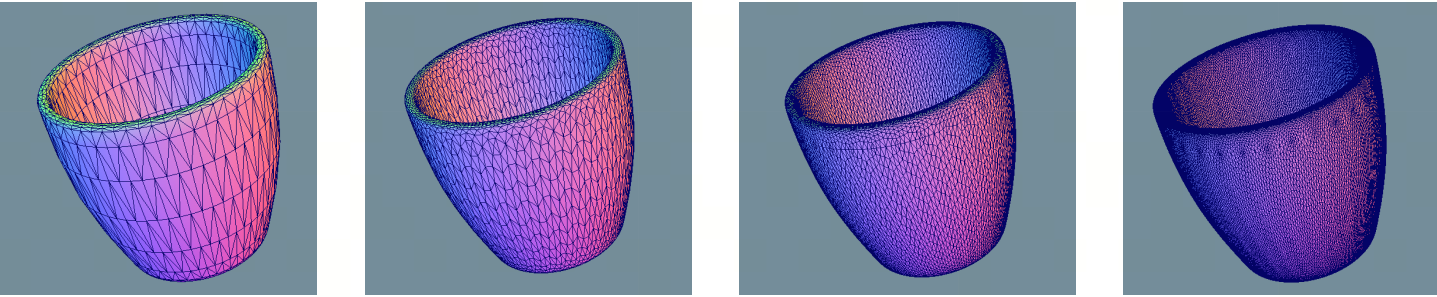
对于原始网格中的每个面，执行以下操作：

获取核心顶点：提取该面的三个原始顶点更新后的新位置 (v'_0, v'_1, v'_2) 及该面生成的中心顶点 c ；

创建新面：删除原始面，基于更新后的顶点创建三个新的三角形面，确保所有面保持逆时针 (CCW) 顺序。

全局拓扑整合：梳理所有新生成的顶点和面，建立完整的网格拓扑关系，完成一次细分迭代。

最终效果如下：



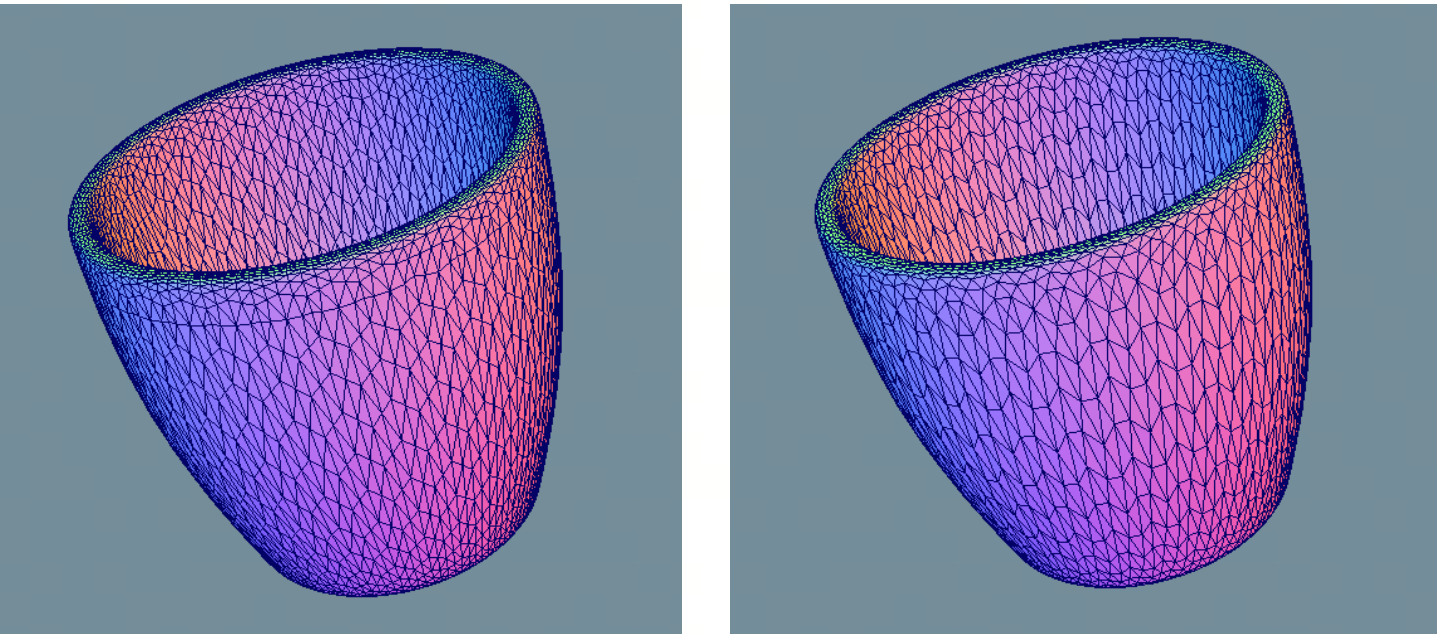
五、对比实验

两个算法中Loop细分采用“顶点分裂”机制，而 $\sqrt{3}$ 细分采用“面分裂+顶点更新”机制：

Loop 细分和 $\sqrt{3}$ 细分在**拓扑变化**上差异很明显。Loop 细分每进行一轮，三角形数量都会变成原来的 4 倍，网格密度提升非常明显；而 $\sqrt{3}$ 细分每轮只把三角形数量变成原来的 3 倍，只在每个面里新增一个中心点，保留了原始顶点和边的整体分布，拓扑结构看起来更简洁。

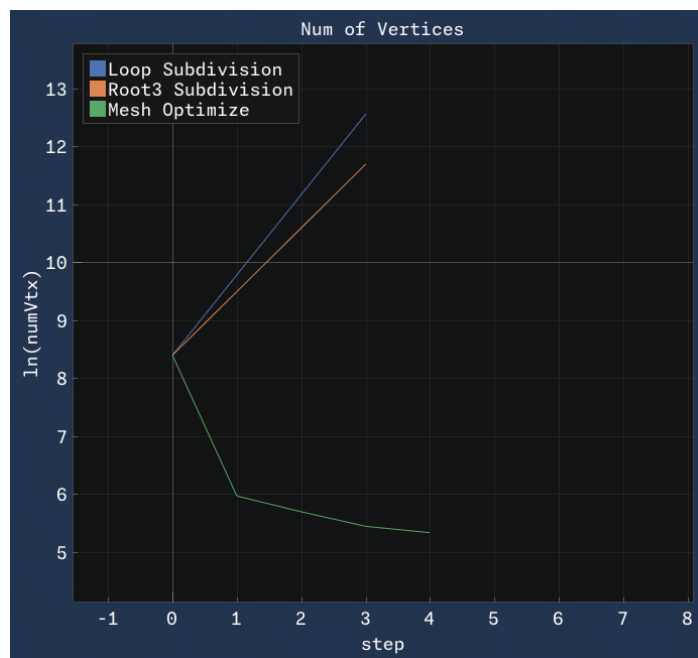
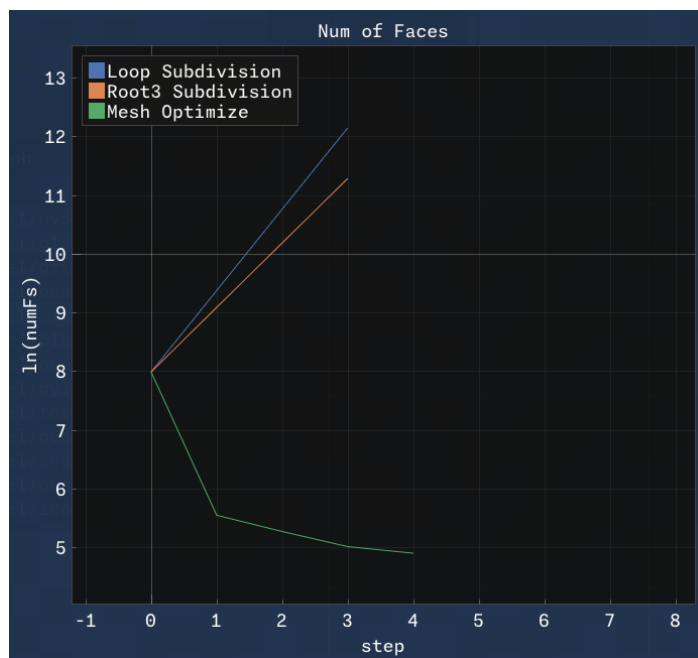
在**平滑效果**上，Loop 细分由于网格更密，平滑得更快也更细腻，能迅速消除棱角，曲面过渡自然、细节更丰富，但在边缘区域有时会出现轻微收缩； $\sqrt{3}$ 细分的平滑过程相对温和，曲面变光滑的速度较慢，不过能更好地保持原始模型的大致形态，不太容易出现边缘收缩。

在**边界处理**方面，Loop 细分需要单独处理边界顶点和边界边，通过加权更新和生成边中点，使边界最终形成平滑曲线；而 $\sqrt{3}$ 细分的边界处理更简单，通常不拆分边界边，只调整边界顶点的位置，因此边界形状基本保持原样，整体稳定性更强。



每轮细分中，loop变为原本四倍，而根号三变为原本三倍，从框架中自带的画图的斜率可以看出差别：

这里还画了一下框架中自带的**网格优化算法**，可以看到优化到后面下降趋势渐小，后面面少了不好减面优化了。



心得

本次实验时间都在看框架，要实现的算法本身并不难，但是一开始没留意框架里面自带的辅助函数，导致一开始写的时候写错了，后面把自实现的改成自带的函数之后就效果很好啦。